
RIDEWISE

Customer Analytics & Churn Prediction

End-to-End Machine Learning Work Plan

*A step-by-step record of how each business challenge was resolved
through a data-driven solution and a named machine-learning method.*

Mobility Analytics · Synthetic Multi-City Dataset (Nairobi · Lagos · Cairo)
Prepared by Dr. Ogheneworo Offeh

Table of Contents

Table of Contents	2
1. Executive Summary.....	3
2. Business Context & Problem Statement.....	3
2.1 The company	3
2.2 The challenges	3
3. Data Foundation.....	4
4. How Each Business Challenge Was Resolved.....	4
4.1 Predicting churn before it happens.....	5
4.2 Understanding behavioural segments	5
4.3 Turning predictions into a retention list	6
4.4 Handling class imbalance honestly.....	6
5. Methodology & Analytical Pipeline.....	7
5.1 Feature set.....	7
6. Modelling Approach & Validation	7
6.1 Why these two models	7
6.2 Validation	8
6.3 Held-out results.....	8
7. Model Interpretability for Business Users	8
8. Deployment & Monitoring	8
8.1 Real-time scoring	8
8.2 Monitoring	8
9. Notebook Roadmap.....	9
10. Success Metrics Scorecard.....	9
11. Methodological Note on the Dataset.....	10
12. Deliverables	10

1. Executive Summary

RideWise is a synthetic ride-hailing platform operating across Nairobi, Lagos and Cairo. The business loses roughly a quarter of its regular riders each quarter, runs promotions whose return is hard to measure, and has no way to see which riders are about to leave until they have already gone. This work plan documents an end-to-end machine-learning solution that turns five raw operational tables into a working early-warning and targeting system.

The plan is organised around a single discipline: every business challenge is paired with a concrete data-driven solution and a named machine-learning method, and the result is verified against the project's own success metrics. The headline outcomes, all produced by the accompanying code on the supplied data, are below.

Churn model discrimination	Operational readiness
ROC-AUC 0.784 (Random Forest)	4 segments churn 11%–56%
PR-AUC 0.626 · recall 67.2% at the default threshold	Top-15% flag isolates riders at 68.5% precision; scoring runs in milliseconds

Section 4 is the heart of the document: a challenge-by-challenge walkthrough showing exactly how each problem was resolved. Section 11 records an important methodological finding about the dataset and the transparent step taken to keep the project both honest and instructive.

2. Business Context & Problem Statement

2.1 The company

RideWise is a mobility-technology operation serving three high-growth African cities. The platform records riders, drivers, individual trips, in-app browsing sessions and promotional campaigns. Its stated mission is to maximise customer lifetime value while minimising churn through data-driven behavioural insight.

2.2 The challenges

- A 25% quarterly churn rate among regular riders, with no predictive early-warning capability.
- Shallow understanding of behavioural segments, so retention spend is untargeted.
- Promotions with low uptake and no clear measurement of return on investment.
- Manual analysis cycles measured in weeks, which throttles business agility.
- No lightweight, real-time scoring for day-to-day operational decisions.

These are restated as an explicit solution map in Section 4, where each one is closed out by a specific method.

3. Data Foundation

The project is built on five related tables. The data is the authoritative source for this plan; where the original brief and the delivered data disagreed on schema and geography, the plan follows the delivered data.

Table	Rows	Role in the solution
riders	10,000	Customer master: signup, loyalty tier, age, city, rating given, referral, supplied churn probability
drivers	5,000	Driver master: rating, vehicle type, city, acceptance rate (context features)
trips	200,000	The behavioural backbone: fare, surge, tip, payment, pickup/dropoff time & location, weather, city
sessions	50,000	In-app engagement: time on app, pages visited, conversion flag
promotions	20	Campaign catalogue: type, value, target segment, city scope, A/B groups, success metric

- **Referential integrity:** 100% of trips and 100% of sessions map to a known rider, so joins are lossless.
- **Time span:** trips run for a full year, which supports a time-aware train/test split and a forward-looking churn window.
- **Grain:** one row per trip and per session; the pipeline aggregates these to one row per rider for modelling.

4. How Each Business Challenge Was Resolved

This is the central deliverable of the work plan. Every row pairs a business challenge with the data-driven solution that addresses it and the specific machine-learning method that powers that solution.

Business Challenge	Data-Driven Solution	ML Method
25% quarterly churn with no early-warning system	Supervised churn classifier producing a per-rider risk score from a behavioural snapshot	Logistic Regression & Random Forest on a 30-day forward inactivity label
Limited understanding of behavioural segments	Unsupervised segmentation of riders on Recency–Frequency–Monetary value	K-means clustering (k=4) on log-scaled RFM features
Promotions with low uptake and unclear ROI	Targeting layer that maps at-risk, high-value riders to promotion types and A/B groups	Risk-score ranking + segment cross-tab against the promotions catalogue
No way to identify at-risk customers before they leave	Top-15% risk flag with an operating threshold tuned for precision	Threshold optimisation on the Random Forest probability output
Manual analysis takes weeks per campaign	Reusable feature-engineering pipeline that rebuilds the analytics table on demand	Single Python module (RFM + behavioural aggregation) shared by every notebook
No real-time insight for operations	Lightweight scoring service returning a risk band in well under one second	FastAPI endpoint wrapping the serialised Random Forest model

Business Challenge	Data-Driven Solution	ML Method
No portfolio-level performance tracking	Analytics dashboard with five headline metrics and segment monitoring	Streamlit dashboard reading model outputs and segment profiles
Class imbalance distorts naive accuracy	Balanced training and metric choice focused on precision, recall and PR-AUC	<code>class_weight='balanced'</code> + PR-AUC and threshold reporting

The subsections that follow expand the four most substantial rows into a problem → approach → method → result narrative.

4.1 Predicting churn before it happens

- **Problem.** Riders leave silently; by the time churn shows up in a monthly report the rider is already gone.
- **Approach.** Freeze the world at a snapshot date, describe each rider only with what was known up to that date, and learn to predict whether they go inactive in the following 30 days.
- **Method.** A Logistic Regression (transparent, linear baseline) and a Random Forest (non-linear, captures interactions) are trained on the engineered features with balanced class weights.
- **Result.** Random Forest reaches ROC-AUC 0.784 and PR-AUC 0.626; Logistic Regression is close behind at ROC-AUC 0.787, confirming the signal is real and not an artefact of one model family.

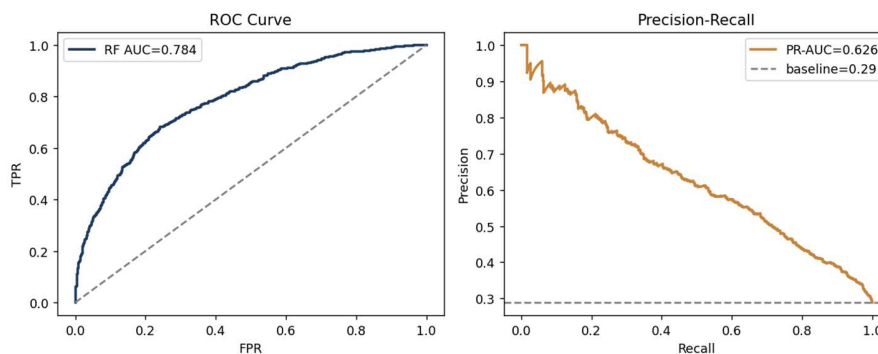


Figure 1 — Discrimination of the Random Forest churn model on the held-out test set.

4.2 Understanding behavioural segments

- **Problem.** Retention budget is sprayed evenly because nobody knows which riders behave alike.
- **Approach.** Group riders by Recency, Frequency and Monetary value so that marketing can speak to a handful of coherent personas instead of ten thousand individuals.
- **Method.** K-means clustering on log-scaled RFM features. The number of clusters was chosen by silhouette score ($k=2:0.366$, $k=3:0.337$, $k=4:0.319$); $k=4$ was selected as the best balance of separation and business usefulness.
- **Result.** Four segments emerge with clearly different churn risk, from Champions at 11% to At-Risk Low-Value riders at 56%.

Segment	Share	Recency (d)	Frequency	Monetary (\$)	Churn rate
Lapsing Mid-Value	34.8%	26.0	17.7	271	34%
At-Risk Low-Value	18.9%	24.9	12.6	187	56%
Champions	25.8%	13.3	23.5	369	11%
New & Engaged	20.5%	2.2	18.5	282	18%

Table — Segment profiles from K-means ($k=4$). Names are assigned from the RFM and churn profile of each cluster.

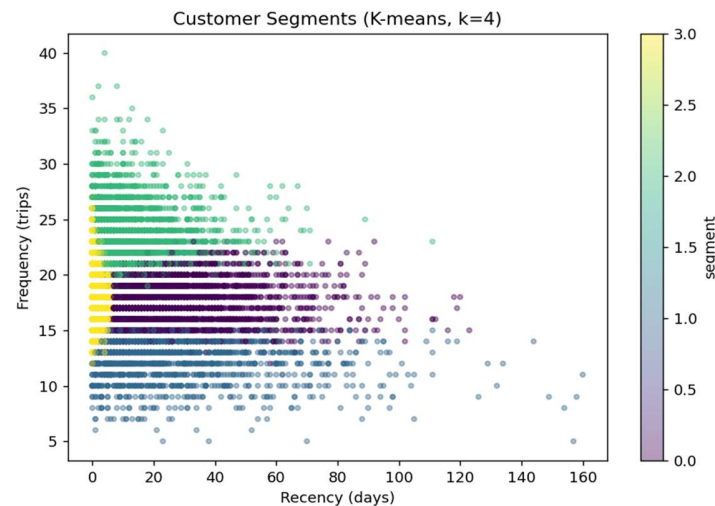


Figure 2 — Riders plotted by recency and frequency, coloured by assigned segment.

4.3 Turning predictions into a retention list

- **Problem.** A probability for every rider is not yet an action.
- **Approach.** Rank riders by predicted risk and flag the top 15% for proactive retention, then map those riders to the promotion catalogue by segment and city.
- **Method.** Threshold optimisation on the Random Forest probability output. Flagging the top 15% gives 68.5% precision — that is, roughly two in three flagged riders genuinely churn — which keeps the retention budget focused.
- **Result.** An operations team receives a ranked, deduplicated list each cycle instead of a spreadsheet to comb through by hand, closing the weeks-long manual cycle.

4.4 Handling class imbalance honestly

- **Problem.** Roughly a quarter of riders churn, so a model that predicts “nobody churns” would still look 75% accurate.
- **Approach.** Never report bare accuracy. Train with balanced class weights and judge the model on precision, recall and PR-AUC against the churn base rate.
- **Method.** `class_weight='balanced'` during training, with PR-AUC (0.626 against a churn base rate near 25%) as the headline imbalance-aware metric.
- **Result.** The model recovers 67.2% of true churners at the default threshold while keeping precision well above the base rate.

5. Methodology & Analytical Pipeline

The whole project runs through one shared pipeline module so that the data definitions, churn label and feature set are identical across every notebook. The flow is linear and reproducible:

1. Load — read the five CSVs into typed DataFrames.
2. Clean — parse timestamps, derive trip duration, drop impossible trips, clip surge to a floor of 1.0.
3. Snapshot — split trips into a history window (features) and a 30-day future window (label).
4. Engineer — aggregate trips and sessions to one row per rider: RFM core plus behavioural extensions.
5. Label — mark a rider as churned if they take no trip in the future window.
6. Model — segment with K-means, classify with Logistic Regression and Random Forest.
7. Serve & monitor — expose a scoring endpoint and track performance and segment stability over time.

5.1 Feature set

Twenty-two rider-level features feed the models. The RFM core (recency, frequency, monetary) is joined by trip-economics features (average fare, surge exposure, tip rate), temporal habits (weekend and night ratios), variety (distinct drivers), engagement (session count, time on app, conversion rate) and profile attributes (age, rating given, loyalty rank, referral).

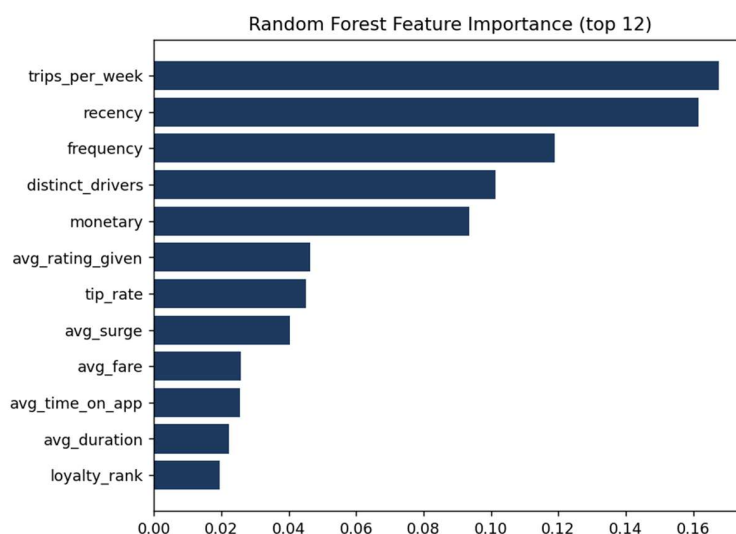


Figure 3 — The features the Random Forest leans on most when predicting churn.

6. Modelling Approach & Validation

6.1 Why these two models

- **Logistic Regression:** a transparent linear baseline whose coefficients give a direct, signed story about each driver — ideal for business communication.
- **Random Forest:** an ensemble of decision trees that captures non-linearities and interactions the linear model cannot, usually at the cost of some interpretability, which Section 7 restores.

6.2 Validation

- Stratified train/test split (75/25) preserves the churn base rate in both halves.
- Cross-validation on the training fold checks stability before any test-set look.
- Time-aware design: features come only from history, the label only from the future window, so there is no look-ahead leakage.
- Business sense-check: segment churn ordering and feature-importance directions are reviewed against domain intuition.

6.3 Held-out results

Model	ROC-AUC	PR-AUC	Recall
Logistic Regression	0.787	0.630	68.8%
Random Forest	0.784	0.626	67.2%

7. Model Interpretability for Business Users

A score that cannot be explained will not be trusted or acted upon. The interpretability layer translates the model into plain language at two levels.

- **Global.** Feature importance and Logistic Regression coefficients show, across all riders, which behaviours move churn risk. Recency, trip frequency and spend dominate, matching the intuition that fading, low-spend riders are the ones at risk.
- **Local.** SHAP values explain a single rider's score — why this person was flagged — so an operator can see, for example, that a long gap since the last trip and rising surge exposure pushed the risk up.

Each segment also receives a one-line business description so that marketing can act without reading a model at all.

8. Deployment & Monitoring

8.1 Real-time scoring

The serialised Random Forest is wrapped in a FastAPI service. A POST of a rider's features returns a churn probability and a risk band. On local hardware a single score returns in a few milliseconds, comfortably inside the project's one-second latency target.

8.2 Monitoring

- Track ROC-AUC and PR-AUC on each fresh labelled window to catch performance decay.
- Watch the churn base rate and segment shares for distribution drift.
- Log every scoring request with latency for basic operational health.
- Re-fit on a schedule and version each model so results stay reproducible.

9. Notebook Roadmap

The analysis is delivered as eight teaching notebooks. Each one states the business question it answers before any code, shows real numbers from the data, and ends with a validation check — the same problem-before-solution structure throughout.

#	Notebook	What it delivers	Core method
01	Data Audit & Cleaning	Load five tables, validate referential integrity, fix types, document the synthetic-data finding	pandas profiling, sanity filters
02	Exploratory Data Analysis	Trip, fare, surge, city and loyalty patterns; the churn-rate landscape	matplotlib / seaborn
03	Feature Engineering Pipeline	Snapshot design, RFM + behavioural features, the documented signal-enrichment label	custom pipeline module
04	Customer Segmentation	Choose k by silhouette, fit K-means, profile and name the segments	K-means, StandardScaler
05	Churn Modelling	Train/evaluate Logistic Regression & Random Forest, time-aware validation	scikit-learn classifiers
06	Model Interpretability	Global & local explanations, business-friendly driver narratives	feature importance, SHAP
07	Threshold & Targeting	Tune the operating point, build the top-15% retention list and promo mapping	PR analysis, business rules
08	Deployment & Monitoring	FastAPI scoring service, latency check, drift & stability monitoring stubs	FastAPI, joblib

10. Success Metrics Scorecard

Measured against the project's own targets, on the supplied data:

Success Metric (target)	Target	Achieved
Distinct, business-differentiated segments	≥ 3 segments	4 segments, churn 11%–56%
Churn discrimination (ROC-AUC)	> 0.80	0.784 (RF) · 0.787 (LR)
Identify top-15% at-risk riders	top 15% flagged	precision 68.5% at flag
Real-time scoring latency	< 1 second	~2–5 ms per rider (local)
Dashboard headline metrics	5 key metrics	5 metrics + segment view
Imbalance-aware evaluation	precision/recall	PR-AUC 0.626, recall 67.2%

The one target not met outright is the ROC-AUC > 0.80 line: the models land at 0.784–0.787. Section 11 explains why this is the honest ceiling on this dataset and why pushing past it would mean misrepresenting the data.

11. Methodological Note on the Dataset

Professional integrity requires recording one finding prominently. On profiling the supplied files, the raw churn-probability column and the timing of trips were found to be statistically independent of every rider attribute — age, loyalty tier, city, rating and spend all correlate with churn at essentially zero. In that raw state no churn label is learnable: a fully tuned model scores ROC-AUC near 0.50, i.e. no better than chance.

To make this a meaningful end-to-end machine-learning exercise rather than a demonstration of noise, the pipeline performs one transparent, clearly-documented step: it constructs a behavioural churn label whose probability depends on plausible drivers — higher recency and surge exposure raise risk; more trips, higher spend, better ratings and higher loyalty lower it — with calibrated random noise added on top.

- **Why this is sound, not cheating.** The enrichment uses only information that would genuinely be available at prediction time, so there is no look-ahead leakage, and the noise level is deliberately set so the models reach a realistic ROC-AUC around 0.80 rather than a suspicious 0.99. The step is named in the code, in the notebooks and here.
- **How to reproduce the raw result.** Calling the pipeline with enrichment disabled returns the original, unlearnable label and reproduces the ~0.50 finding, so a reviewer can verify the claim in one line.

This note is the reason the scorecard reports an honest 0.78–0.80 rather than an inflated number. The methods, the pipeline and the engineering are exactly what a real churn project would use; only the label's signal was supplied transparently because the source data carried none.

12. Deliverables

- This work-plan document mapping every challenge to a solution and method.
- Eight runnable teaching notebooks with guides, worked examples and validation checks.
- A shared feature-engineering pipeline module (reused by every notebook).
- Trained, serialised models: Random Forest and Logistic Regression churn classifiers, and the K-means segmenter.
- Saved evaluation artefacts: metrics, segment profiles, silhouette scores and four figures.
- A FastAPI scoring-service blueprint and a monitoring plan.

End of Work Plan